LARSEN & TOUBRO

Working with Files

1. Reading and Writing Plain Text Files

To open a file, use the built-in open() function. Always use a with statement (a context manager); it automatically closes the file for you, even if an error occurs.

Writing Text ('w' and 'a')

- 'w' mode overwrites the file entirely.
- 'a' mode appends new data to the end of the existing file.

```
# Writing to a file (creates the file if it doesn't exist)
with open("example.txt", "w", encoding="utf-8") as file:
    file.write("Hello, World!\n")
    file.write("This is a second line of text.")
```

```
with open("example.txt", "a", encoding="utf-8") as file:
    file.write("\nThis line was appended.")
```

Reading Text ('r')

- .read() reads the entire file as a single string.
- .readlines() reads all lines into a list of strings.
- Iterating directly over the file object reads line-by-line (memory efficient for large files).

```
with open("example.txt", "r", encoding="utf-8") as file:
    content = file.read()
    print(content)
```

```
with open("example.txt", "r", encoding="utf-8") as file:
    for line in file:
        print(line.strip())                    # .strip() removes trailing newline characters
```

2. Working with CSV (Comma-Separated Values) Data

Python includes a built-in csv module to handle tabular data easily.

Writing to a CSV File

```
import csv
data = [
    ["Name", "Age", "Profession"],
    ["Alice", 30, "Engineer"],
    ["Bob", 25, "Designer"],
    ["Charlie", 35, "Manager"],
]
with open("people.csv", "w", newline="", encoding="utf-8") as file:
    writer = csv.writer(file)
    writer.writerows(data)
```

Reading from a CSV File

Using csv.DictReader maps each row to a dictionary using the header row as keys, which makes your code much easier to read.

```
import csv
with open("people.csv", "r", encoding="utf-8") as file:
    reader = csv.DictReader(file)
    for row in reader:
        print(f"{row['Name']} is {row['Age']} years old and works as a {row['Profession']}.")
```

3. Working with JSON (JavaScript Object Notation) Data

JSON is the standard format for configuration files and API responses. Python's built-in json module translates JSON directly into Python dictionaries and lists.

- json.dump() / json.dumps(): Write/serialize Python objects to JSON.
- json.load() / json.loads(): Read/deserialize JSON into Python objects.

Writing JSON Data

```
import json
data = {
   "user": "Alice",
   "id": 1042,
   "active": True,
   "skills": ["Python", "SQL", "Git"],
}
with open("config.json", "w", encoding="utf-8") as file:
   json.dump(data, file, indent=4)
```

Reading JSON Data

```
import json
with open("config.json", "r", encoding="utf-8") as file:
   data = json.load(file)
print(data["user"])         # Alice
print(data["skills"])       # ['Python', 'SQL', 'Git']
```

Summary Cheat Sheet

Format	Module	Write Function	Read Function	Best Used For
Text	Built-in	file.write()	file.read() iteration	Logs, notes, code, raw text
CSV	csv	csv.writer().writerow()	csv.DictReader()	Spreadsheets, tabular datasets
JSON	json	json.dump()	json.load()	Configs, nested data, APIs

The Modern Way: You don't call close() manually

When you use a with statement (a context manager), Python **automatically calls close() for you** as soon as the indented block of code finishes—even if an error or exception occurs inside the block. This is why it is the standard and recommended way to handle files in Python.

```
# close() is handled automatically here
with open("example.txt", "r", encoding="utf-8") as file:
    content = file.read()
```

The Traditional Way: If you omit with, close() *is* mandatory

If you open a file using just the open() function without a with statement, you **must** call file.close() explicitly.

```
# You must manually close the file here
file = open("example.txt", "r", encoding="utf-8")
content = file.read()
file.close()                                    # Mandatory if you want to avoid resource leaks
```

Excel and CSV Automation

Automating Excel and CSV tasks in Python transforms tedious, error-prone manual spreadsheets into lightning-fast, repeatable processes. Whether you are merging dozens of sales reports, cleaning messy data, or generating polished executive dashboards, Python has powerful libraries built for the job.

1. The Core Python Libraries

Depending on whether you are working with simple comma-separated files or multi-tab Excel workbooks with formulas and formatting, you will rely on two primary libraries:

- **csv (Built-in):** Perfect for reading and writing basic comma-separated or tab-separated text files. No installation needed.
- **pandas (Third-party):** The gold standard for data manipulation, cleaning, aggregation, and merging across multiple files or large datasets.
- **openpyxl (Third-party):** Specifically designed for reading, writing, and styling .xlsx files—allowing you to add formulas, cell formatting, charts, and multiple sheets without needing Microsoft Excel installed.

2. Automating CSV Tasks with pandas

When dealing with large datasets or tasks like filtering rows, calculating summaries, or combining multiple CSVs, pandas is unmatched.

Example 1: Reading and Filtering a CSV File

Imagine you have a CSV file named employees.csv with employee details, and you want to filter out everyone who earns less than $50,000 and save them to a new file.

Input (employees.csv):

```
Name,Department,Salary
Alice,Engineering,75000
Bob,Marketing,45000
Charlie,Engineering,90000
Diana,HR,48000
```

Python Code (using pandas):

```
import pandas as pd
df = pd.read_csv("employees.csv")
high_earners = df[df["Salary"] >= 50000]
high_earners.to_csv("high_earners.csv", index=False)
print("Filtered data saved successfully!")
print(high_earners)
```

Example: Combining Multiple CSV Reports into One

Imagine you have monthly sales files (jan.csv, feb.csv, etc.) and want to merge them and calculate total sales by region:

```
import pandas as pd

import glob

all_files = glob.glob("sales_*.csv")

df_list = [pd.read_csv(file) for file in all_files]

combined_df = pd.concat(df_list, ignore_index=True)

summary_df = combined_df.groupby("Region")["Revenue"].sum().reset_index()

summary_df.to_csv("annual_sales_summary.csv", index=False)

print("Reports successfully merged and summarized!")
```

3. Automating Excel Workbooks with openpyxl

If you need to generate professional spreadsheets complete with formulas, custom column widths, alternating row colors (zebra striping), or multiple tabs, openpyxl gives you pixel-level control.

Example: Creating a Simple Excel Spreadsheet

This script creates a new Excel file, adds a tiny expense list, and calculates a total using an Excel formula.

```
import openpyxl

wb = openpyxl.Workbook()

ws = wb.active

ws.title = "Expenses"

ws["A1"] = "Item"

ws["B1"] = "Cost"

expenses = [

    ["Rent", 1200],

    ["Utilities", 150],

    ["Internet", 60],

]

for row_idx, item in enumerate(expenses, start=2):

    ws.cell(row=row_idx, column=1, value=item[0])

    ws.cell(row=row_idx, column=2, value=item[1])

ws["A5"] = "Total"

ws["B5"] = "=SUM(B2:B4)"

wb.save("monthly_expenses.xlsx")

print("Excel spreadsheet created successfully!")
```

Example: Modifying an Existing Excel File

If you already have an Excel file and just want to update a specific cell value (for example, updating a project status), you can load it, modify it, and save it back.

```
import openpyxl

wb = openpyxl.load_workbook("monthly_expenses.xlsx")

ws = wb["Expenses"]

ws["B2"] = 1250

ws.append(["Groceries", 250])

wb.save("monthly_expenses_updated.xlsx")

print("Excel file updated successfully!")
```

Data Analysis (Pandas)

Data analysis with **pandas** is all about taking raw, messy data and turning it into clean, actionable insights. Pandas gives you powerful tools to inspect, clean, filter, and summarize datasets with just a few lines of code.
Here is a breakdown of the core workflow: **Cleaning**, **Filtering**, and **Summarizing**.

1. Setting Up and Inspecting Data

Before cleaning data, you need to load it and see what you are working with.

```
import pandas as pd
df = pd.read_csv("sales_data.csv")
print(df.head())          # View the first 5 rows
print(df.info())          # Check column names, data types, and missing values
print(df.describe())      # Get statistical summary for numeric columns
```

2. Cleaning Data (Fixing Messy Real-World Data)

Real-world data is rarely clean—it often has missing values, incorrect types, or duplicate rows.

Handling Missing Values

```
# Drop rows where any column has missing values
df_cleaned = df.dropna()
```

```
# Fill missing values (e.g., replace missing ages with the average age)
df["Age"] = df["Age"].fillna(df["Age"].mean())

# Fill missing text/categories with a default value
df["Department"] = df["Department"].fillna("Unknown")
```

Removing Duplicates

```
# Check for duplicate rows and remove them
df = df.drop_duplicates()
```

Fixing Data Types

```
# Convert a date column from string to actual datetime objects
df["Order_Date"] = pd.to_datetime(df["Order_Date"])

# Convert a currency column string (e.g., "$1,200") into numbers
df["Price"] = df["Price"].str.replace("$", "").str.replace(",", "").astype(float)
```

3. Filtering Data (Finding Exactly What You Need)

Filtering allows you to isolate specific subsets of your data using conditional statements.

```
# 1. Filter rows where Region is 'North'
north_sales = df[df["Region"] == "North"]

# 2. Filter with multiple conditions (AND: &) - Sales > 500 AND Category is 'Electronics'
top_electronics = df[(df["Sales"] > 500) & (df["Category"] == "Electronics")]

# 3. Filter with multiple conditions (OR: |) - Region is 'East' OR 'West'
east_west = df[(df["Region"] == "East") | (df["Region"] == "West")]

# 4. Filter using .isin() for multiple options in a list
allowed_regions = ["North", "South", "East"]
filtered_regions = df[df["Region"].isin(allowed_regions)]
```

4. Summarizing Data (Aggregating and Grouping)

Once your data is clean and filtered, you can aggregate it to find totals, averages, and trends.

Basic Summary Statistics

```
total_sales = df ["Sales"].sum()
average_price = df ["Price"].mean()
max_quantity = df ["Quantity"].max()
```

Grouping Data (groupby)

The groupby function is the pandas equivalent of an SQL GROUP BY or an Excel Pivot Table. It splits data into groups and calculates metrics for each.

```
# Find total sales revenue per region
sales_by_region = df.groupby("Region")["Sales"].sum().reset_index()
print(sales_by_region)
# Calculate both total sales and average quantity per category
summary_stats = df.groupby("Category").agg(
    Total_Sales=("Sales", "sum"),
    Average_Quantity=("Quantity", "mean")
).reset_index()
print(summary_stats)
```

Operational Insights

Operational insights are quick, data-driven answers to business questions that help managers and decision-makers take immediate action. Instead of just looking at raw numbers, operational insights answer questions like:

- *Which products are dragging down our profit margins?*
- *Which regions are underperforming this month compared to last month?*
- *Who are our top 10% customers driving the majority of our revenue?*

Using **pandas**, you can extract these actionable insights in just a few lines of code.

1. Finding Top Performers (The 80/20 Rule / Pareto Analysis)

Decision context: *Which products or customers should we focus our marketing budget on?*

```
import pandas as pd
df = pd.read_csv("sales_data.csv")
top_products = (
   df.groupby("Product")["Revenue"]
   .sum()
   .reset_index()
   .sort_values(by="Revenue", ascending=False)
)
print("Top 5 Products by Revenue:")
print(top_products.head(5))
```

2. Detecting Anomalies or Bottlenecks (Outlier Detection)

Decision context: *Are there unusual shipping delays or abnormally high expenses that need investigating?*

```
# Find orders where delivery time is significantly higher than average
mean_delivery_days = df["Delivery_Days"].mean()
threshold = mean_delivery_days * 1.5
delayed_orders = df[df["Delivery_Days"] > threshold]
print(f"Alert: {len(delayed_orders)} orders experienced abnormal delays.")
print(delayed_orders[["Order_ID", "Customer", "Delivery_Days"]].head())
```

3. Period-Over-Period Growth (Trend Analysis)

Decision context: *Are sales growing or shrinking compared to the previous period?*

Python

```
# Convert date column and extract month/year
df["Date"] = pd.to_datetime(df["Date"])
df["Month"] = df["Date"].dt.to_period("M")

# Monthly revenue trend
monthly_revenue = df.groupby("Month")["Revenue"].sum().reset_index()
# Calculate percentage change month-over-month
monthly_revenue["Growth_Rate"] = monthly_revenue["Revenue"].pct_change() * 100
print(monthly_revenue)
```

4. Identifying Inactive or Churning Customers

Decision context: *Which valuable customers haven't made a purchase in the last 90 days?*

```
# Find the most recent date in the dataset
latest_date = pd.to_datetime(df["Date"]).max()

# Find the last purchase date for each customer
customer_last_purchase = df.groupby("Customer_ID")["Date"].max().reset_index()

# Calculate days since last purchase
customer_last_purchase["Days_Inactive"]= (latest_date - pd.to_datetime(customer_last_
purchase["Date"])).dt.days

# Filter for customers inactive for more than 90 days
churn_risk = customer_last_purchase[customer_last_purchase["Days_Inactive"] > 90]
print(f"Found {len(churn_risk)} customers at risk of churn.")
```